# Inkcast end-to-end walkthrough — 0.1.0 at f4a9e53ced19

Inkcast 0.1.0 (commit f4a9e53ced196b28244fdeee4f72200cf2587462, main) run in development mode against the fake DEV and Medium APIs and the stand-in Keycloak from the test suite, driven by a real MCP client (the Python SDK, OAuth with PKCE) and Playwright (Chromium 141.0.7390.37, Playwright 1.56.0, Python 3.14.8) through `http://localhost:8787`. Run 2026-10-28T07:11:06Z → 2026-10-28T07:11:25Z. Every number in this document is read from results.json, integrity.jsonl and env.json; nothing is typed in. Generated credentials are masked everywhere; the password is never written.

> **Outcome: PASS.** 42 of 42 steps passed (16 MCP calls, 11 REST requests, 14 browser pages); 11 of 11 integrity checks passed.

## 1. What was measured

| Measure | Value |
|---|---|
| Inkcast server (MCP initialize) | inkcast 0.1.0; 11 tools: inkcast_platforms, inkcast_connect, inkcast_disconnect, inkcast_destinations, inkcast_publish, inkcast_sync, inkcast_posts, inkcast_cancel, inkcast_resend, inkcast_create_token, inkcast_delete_account |
| MCP client | Walkthrough MCP client — dynamic registration, PKCE S256, redirect to http://localhost:44043/callback |
| OAuth token | scope “inkcast”, expires in 3600 s |
| Steps | 42 (42 passed, 0 failed): 16 MCP, 11 REST, 14 browser |
| Pictures | 14 screenshots of Inkcast's pages, 27 rendered panels of MCP and REST calls |
| Draft on DEV | “Publishing from an MCP client” → https://dev.to/ada/article-1 (post pst_565d59c38bc4) |
| Scheduled then cancelled | “A story for later this hour” for 2026-10-28T08:11:12+00:00 (post pst_fc8f71c396ee) |
| Personal access token | ink_pat_ecd734c2ae43… (masked; scope stories:sync) |
| Git sync | github.com/acme/blog:posts/hello-from-a-repository.md → create published https://dev.to/ada/article-2 |
| Ledger entries | 1 |
| Articles on the fake DEV at the end | 2: “Publishing from an MCP client” (draft); “Hello from a repository” (published) |
| Integrity checks | 11 of 11 passed |
| Run | 2026-10-28T07:11:06Z → 2026-10-28T07:11:25Z |

## 2. The journey, step by step

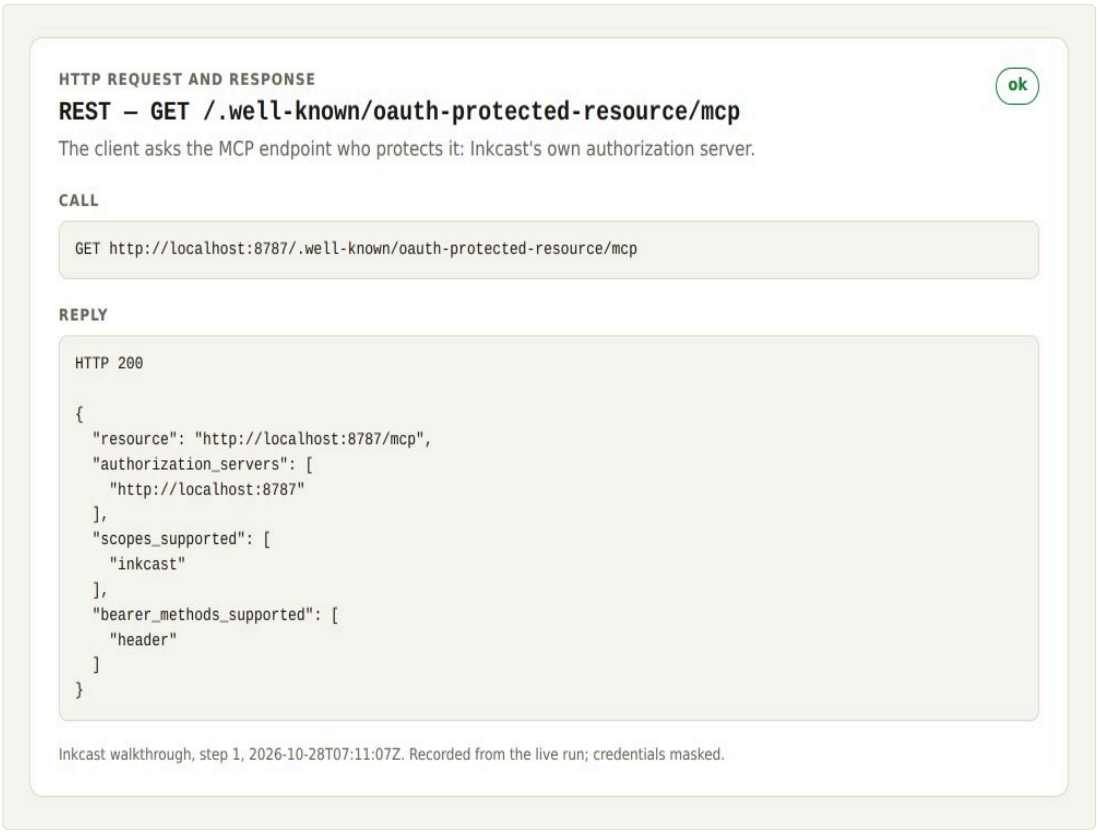
Each step is pictured as it happened: a screenshot of the Inkcast page the person saw, or a panel rendering the MCP tool call or REST request the client made and the reply it got. The line under each picture is what the step checked; a check that fails fails the step. A page with a JavaScript error, an HTTP error status or a visible error is refused rather than pictured as a success.

### 1. Add Inkcast to an MCP client

**Step 1** — REST — GET /.well-known/oauth-protected-resource/mcp (rest, 2026-10-28T07:11:08Z)

The client asks the MCP endpoint who protects it: Inkcast's own authorization server.

✓ HTTP 200; the resource names its authorization server




01-rest-get-well-known-oauth-protected-resource-mcp.png — HTTP 200; the resource names its authorization server

**Step 2** — REST — GET /.well-known/oauth-authorization-server (rest, 2026-10-28T07:11:08Z)

Discovery: where to register, authorize and get tokens; PKCE S256 is required.

✓ HTTP 200; metadata lists registration, authorization and token endpoints with S256

HTTP REQUEST AND RESPONSE

ok

## REST — GET /.well-known/oauth-authorization-server

Discovery: where to register, authorize and get tokens; PKCE S256 is required.

CALL

```
GET http://localhost:8787/.well-known/oauth-authorization-server
```

REPLY

```
HTTP 200

{
  "issuer": "http://localhost:8787",
  "authorization_endpoint": "http://localhost:8787/authorize",
  "token_endpoint": "http://localhost:8787/token",
  "registration_endpoint": "http://localhost:8787/register",
  "scopes_supported": [
    "inkcast"
  ],
  "response_types_supported": [
    "code"
  ],
  "grant_types_supported": [
    "authorization_code",
    "refresh_token"
  ],
  "token_endpoint_auth_methods_supported": [
    "client_secret_post",
    "client_secret_basic"
  ],
  "revocation_endpoint": "http://localhost:8787/revoke",
  "revocation_endpoint_auth_methods_supported": [
    "client_secret_post",
    "client_secret_basic"
  ],
  "code_challenge_methods_supported": [
    "S256"
  ]
}
```

Inkcast walkthrough, step 2, 2026-10-28T07:11:08Z. Recorded from the live run; credentials masked.

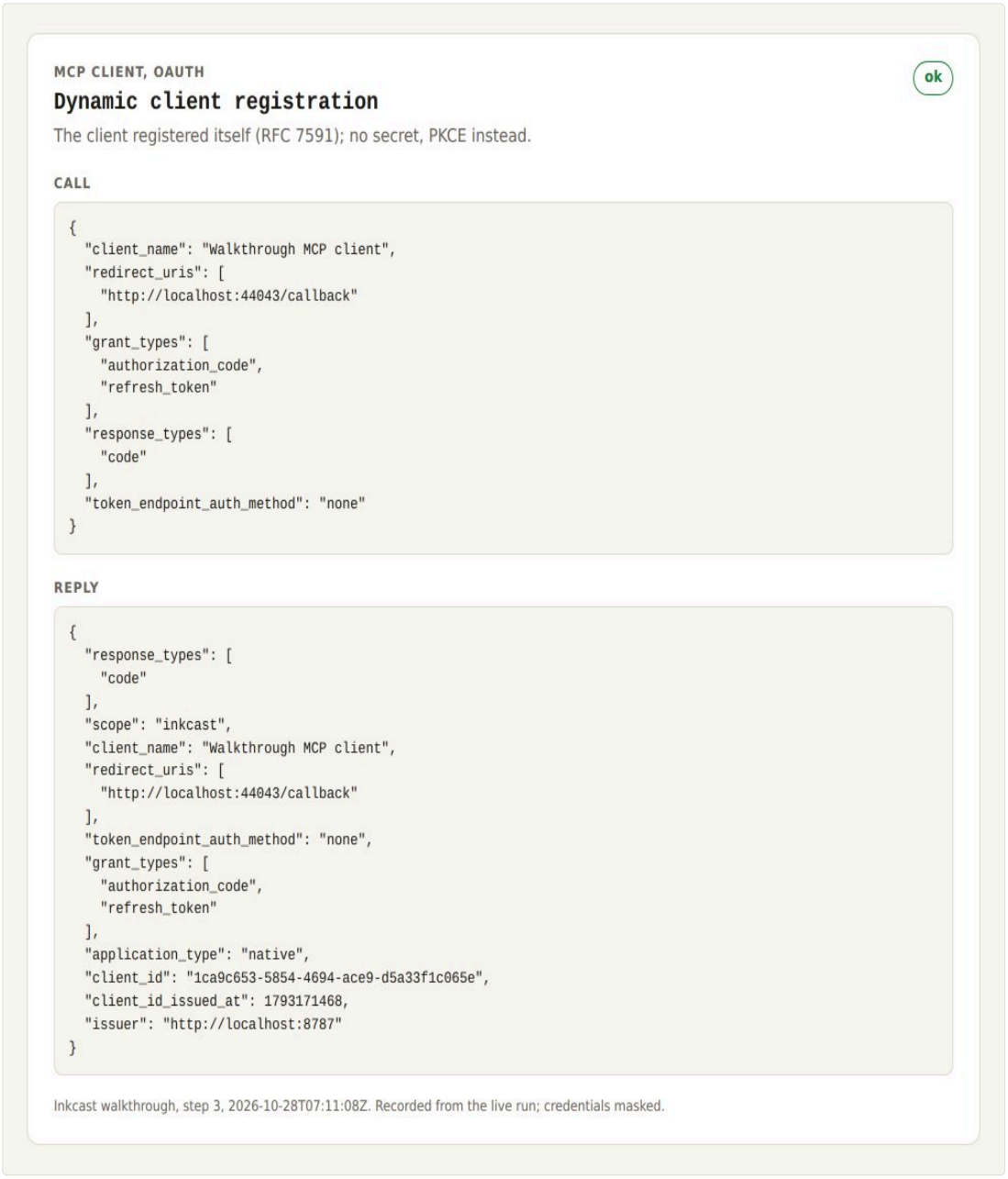
02-rest-get-well-known-oauth-authorization-server.png — HTTP 200; metadata lists registration, authorization and token endpoints with S256

**Step 3** — dynamic client registration (mcp, 2026-10-28T07:11:08Z)

The client registered itself (RFC 7591); no secret, PKCE instead.

✓ registered as “Walkthrough MCP client” (client_id 1ca9c653-5854-4694-ace9-d5a33f1c065e), redirect

http://localhost:44043/callback



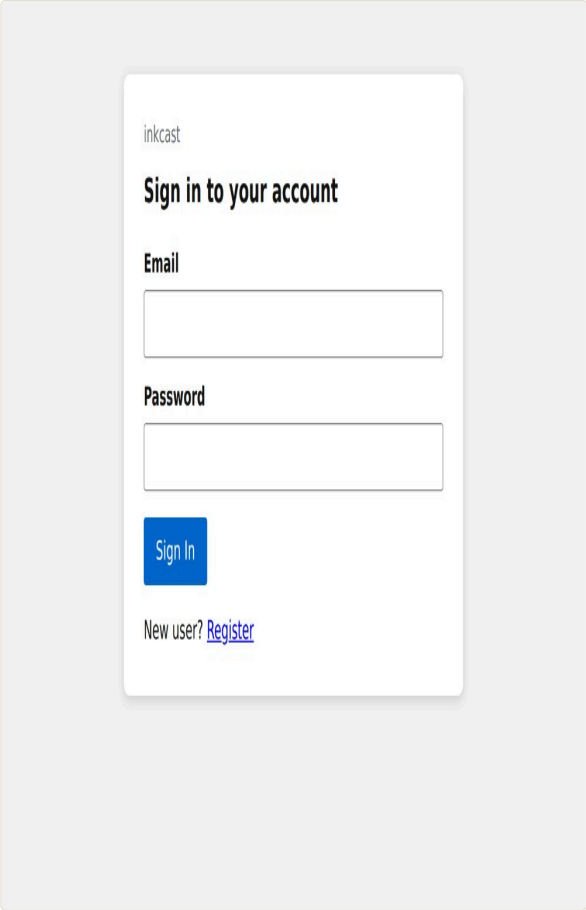

03-dynamic-client-registration.png — registered as “Walkthrough MCP client” (client_id 1ca9c653-5854-4694-ace9-d5a33f1c065e), redirect http://localhost:44043/callback

**Step 4** — sign-in page (browser, 2026-10-28T07:11:09Z)

The client opened the authorization URL in the browser; Inkcast sent it to the identity provider (the stand-in for Keycloak), which asks the person to sign in.

✓ http://127.0.0.1:8999/kc/realms/inkcast/protocol/openid-connect/auth?response_type=code&client_id=inkcast&redirect_uri=http%3A%2F%2Flocalhost%3A8787%2Fauth%2Fcallback&scope=openid+email+pro

BF-Oiskk5KaoNqE9j8tz_hiZcsUg&code_challenge=fd_r5VVYjxmbnc120Z9a7f5FWpaY2eJN1p1j-jWBPpQ&code_challenge_method=S256 — Sign in to your account; the identity provider's sign-in form



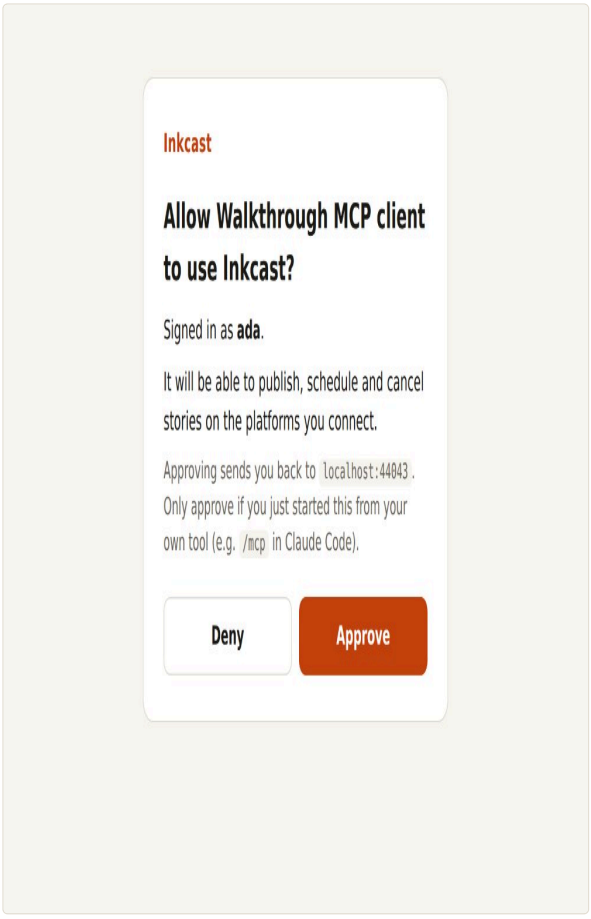

04-sign-in-page.png — http://127.0.0.1:8999/kc/realms/inkcast/protocol/openid-connect/auth?response_type=code&client_id=inkcast&redirect_uri=http%3A%2F%2Flocalhost%3A8787%2Fauth%2Fcallback&scope=openid+email+profile&state=_-7ysWf8TxRj1n0RnpTbuLOcnfSoOZr4&nonce=BF-Oiskk5KaoNqE9j8tz_hiZcsUg&code_challenge=fd_r5VVYjxmbnc120Z9a7f5FWpaY2eJN1p1j-jWBPpQ&code_challenge_method=S256 — Sign in to your account; the identity provider's sign-in form

**Step 5** — consent page (browser, 2026-10-28T07:11:09Z)

Signed in: Inkcast asks whether this client may use the account, naming the client.

✓ http://localhost:8787/auth/consent?pending=_-7ysWf8TxRj1n0RnpTbuLOcnfSoOZr4 — Allow Walkthrough MCP client to use

Inkcast?; consent names the client




05-consent-page.png — http://localhost:8787/auth/consent?pending=_-7ysWf8TxRj1n0RnpTbuLOcnfSoOZr4 — Allow Walkthrough MCP client to use Inkcast?; consent names the client

**Step 6** — authorization code at the local callback (check, 2026-10-28T07:11:10Z)

The browser was sent back to the client's local listener with a one-time code; the client exchanges it with its PKCE verifier.

✓ the redirect reached http://localhost:44043/callback with a code and state

**Step 7** — token exchange (mcp, 2026-10-28T07:11:10Z)

The code and verifier became an access token and a refresh token; the client stores them.

✓ access token issued, expires in 3600 s, scope: inkcast

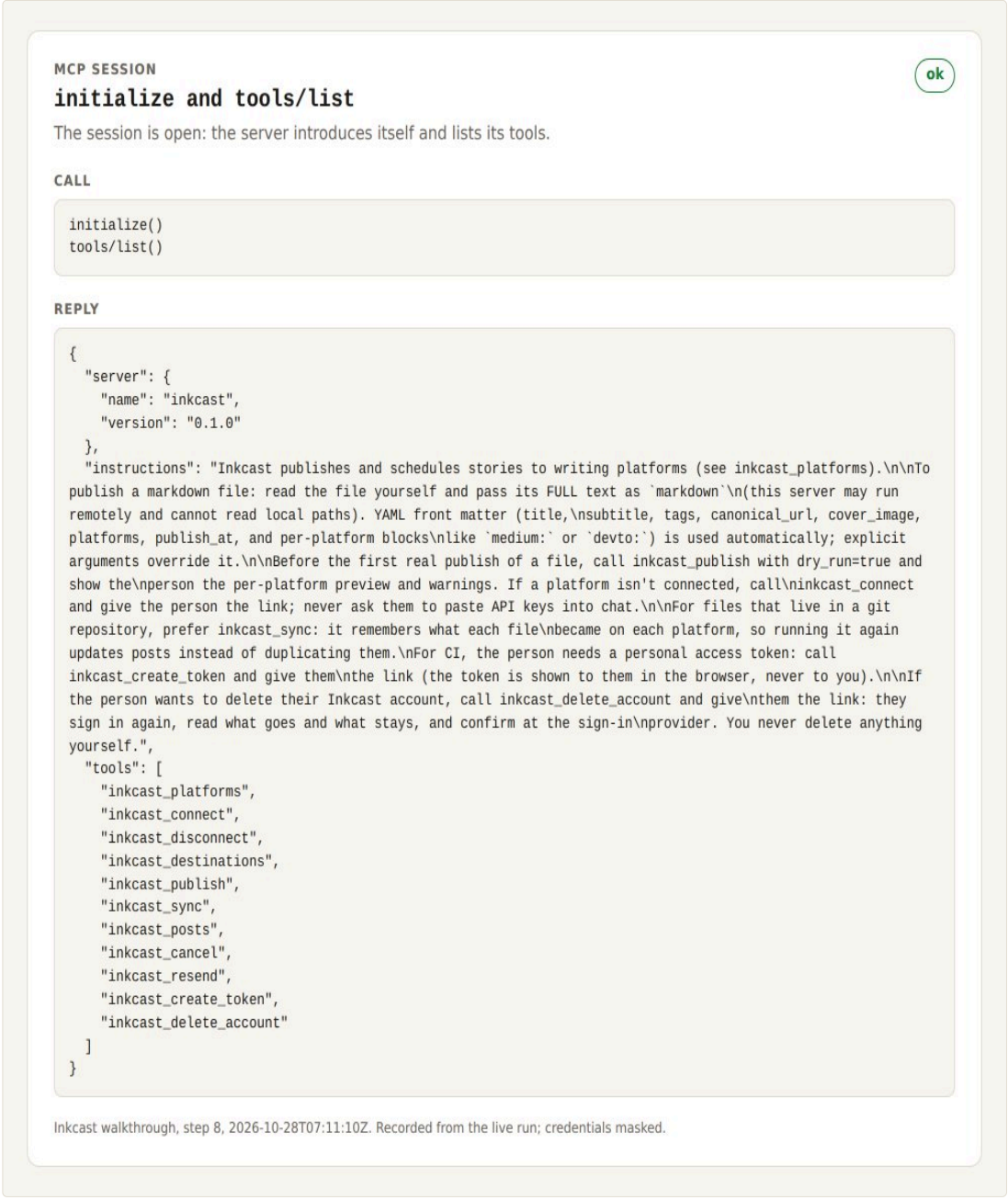



06-token-exchange.png — access token issued, expires in 3600 s, scope: inkcast

**Step 8** — MCP initialize (mcp, 2026-10-28T07:11:10Z)

The session is open: the server introduces itself and lists its tools.

✓ server “inkcast” 0.1.0; 11 tools: inkcast_platforms, inkcast_connect, inkcast_disconnect, inkcast_destinations, inkcast_publish, inkcast_sync, inkcast_posts, inkcast_cancel, inkcast_resend, inkcast_create_token, inkcast_delete_account




07-initialize-and-tools-list.png — server “inkcast” 0.1.0; 11 tools: inkcast_platforms, inkcast_connect, inkcast_disconnect, inkcast_destinations, inkcast_publish, inkcast_sync, inkcast_posts, inkcast_cancel, inkcast_resend, inkcast_create_token, inkcast_delete_account

### 2. Platforms, connect DEV, destinations

**Step 9** — MCP inkcast_platforms (mcp, 2026-10-28T07:11:10Z)

Nothing is connected yet.

✓ inkcast_platforms: DEV and Medium listed, both not connected

MODEL CONTEXT PROTOCOL, TOOLS/CALL

ok

**MCP tool call — inkcast_platforms**

Nothing is connected yet.

CALL

```
inkcast_platforms()
```

REPLY

```
Platforms:
- `devto` — DEV: not connected; supports organizations
- `medium` — Medium: not connected; supports publications
```

Inkcast walkthrough, step 9, 2026-10-28T07:11:10Z. Recorded from the live run; credentials masked.

08-mcp-tool-call-inkcast-platforms.png — inkcast_platforms: DEV and Medium listed, both not connected

**Step 10** — MCP inkcast_connect (mcp, 2026-10-28T07:11:10Z)

The assistant gets a one-time link for the person; the key never goes through chat.

✓ inkcast_connect: a one-time /connect/ link

MODEL CONTEXT PROTOCOL, TOOLS/CALL

ok

**MCP tool call — inkcast_connect**

The assistant gets a one-time link for the person; the key never goes through chat.

CALL

```
inkcast_connect({
  "platform_id": "devto"
})
```

REPLY

```
Open this link to connect DEV (valid 15 minutes, one use):
http://localhost:8787/connect/Gm5h31qUJGHAbsrC3QOwIYtcXqRPcOntkB--kmp3jUo

Paste a DEV API key. It lets Inkcast create articles as you.
```

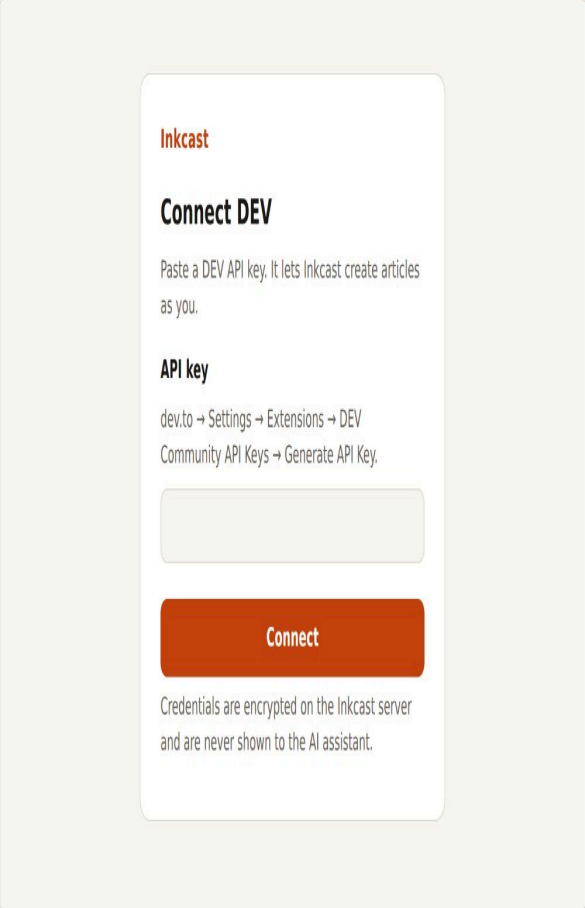
Inkcast walkthrough, step 10, 2026-10-28T07:11:10Z. Recorded from the live run; credentials masked.

09-mcp-tool-call-inkcast-connect.png — inkcast_connect: a one-time /connect/ link

**Step 11** — connect page (browser, 2026-10-28T07:11:11Z)

The person opens the link: a form for the DEV API key, with the help text.

✓ http://localhost:8787/connect/Gm5h31qUJGHAbsrC3QOwIYtcXqRPcOntkB--kmp3jUo — Connect DEV; the DEV connect form



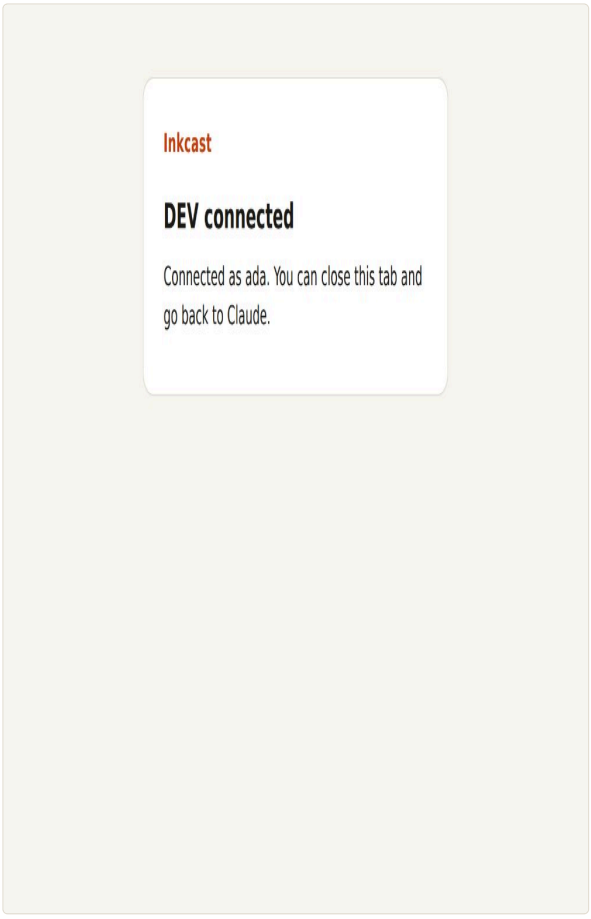

10-connect-page.png — http://localhost:8787/connect/Gm5h31qUJGHAbsrC3QOwIYtcXqRPcOntkB--kmp3jUo — Connect DEV; the DEV connect form

**Step 12** — dev connected (browser, 2026-10-28T07:11:12Z)

Inkcast verified the key with DEV and stored it; the link is used up.

✓ http://localhost:8787/connect/Gm5h31qUJGHAbsrC3QOwIYtcXqRPcOntkB--kmp3jUo — DEV connected; the page says DEV

connected as ada




11-dev-connected.png — http://localhost:8787/connect/Gm5h31qUJGHAbsrC3QOwIYtcXqRPcOntkB--kmp3jUo — DEV connected; the page says DEV connected as ada

**Step 13** — MCP inkcast_platforms (mcp, 2026-10-28T07:11:12Z)

Back in the client: DEV is now connected.

✓ inkcast_platforms: DEV connected as ada




12-mcp-tool-call-inkcast-platforms.png — inkcast_platforms: DEV connected as ada

**Step 14** — MCP inkcast_destinations (mcp, 2026-10-28T07:11:12Z)

Where else DEV can post: organizations, which DEV's API cannot list, so the reply says how to name one.

✓ inkcast_destinations: the reply explains organizations




13-mcp-tool-call-inkcast-destinations.png — inkcast_destinations: the reply explains organizations

### 3. Preview, publish a draft, list posts

**Step 15** — MCP inkcast_publish (mcp, 2026-10-28T07:11:12Z)

A dry run first: DEV's version of the story, with any warnings, and nothing sent.

✓ inkcast_publish: a dry-run preview for DEV

MODEL CONTEXT PROTOCOL, TOOLS/CALL

ok

**MCP tool call — inkcast_publish**

A dry run first: DEV's version of the story, with any warnings, and nothing sent.

CALL

```
inkcast_publish({
  "markdown": "<markdown: 17 lines, shown below>",
  "dry_run": true
})

markdown =
---
title: Publishing from an MCP client
subtitle: One markdown file, every platform, no copy and paste
tags: [writing, publishing, markdown, mcp]
canonical_url: https://blog.example.com/publishing-from-an-mcp-client
platforms: [devto]
---

# Publishing from an MCP client

You write the story once, in markdown, and your assistant publishes it where you asked.
Inkcast keeps the credentials; the assistant only ever sees the link.

## What happens

1. A dry run shows you each platform's version.
2. A draft lets you check it on the platform before it goes live.
```

REPLY

```
DRY RUN — nothing was sent.
“Publishing from an MCP client” — One markdown file, every platform, no copy and paste
[DEV]
  title: Publishing from an MCP client
  description: One markdown file, every platform, no copy and paste
  tags: writing, publishing, markdown, mcp
  canonical_url: https://blog.example.com/publishing-from-an-mcp-client
  status: published
  organization: (your profile)
```

Inkcast walkthrough, step 15, 2026-10-28T07:11:12Z. Recorded from the live run; credentials masked.

14-mcp-tool-call-inkcast-publish.png — inkcast_publish: a dry-run preview for DEV

**Step 16** — MCP inkcast_publish (mcp, 2026-10-28T07:11:12Z)

Then the real thing, as a draft on DEV so it can be checked there first.

✓ inkcast_publish: a DEV draft with its URL and post id

MODEL CONTEXT PROTOCOL, TOOLS/CALL

ok

## MCP tool call — inkcast_publish

Then the real thing, as a draft on DEV so it can be checked there first.

CALL

```
inkcast_publish({
  "markdown": "<markdown: 17 lines, shown below>",
  "draft": true
})

markdown =
---
title: Publishing from an MCP client
subtitle: One markdown file, every platform, no copy and paste
tags: [writing, publishing, markdown, mcp]
canonical_url: https://blog.example.com/publishing-from-an-mcp-client
platforms: [devto]
---

# Publishing from an MCP client

You write the story once, in markdown, and your assistant publishes it where you asked.
Inkcast keeps the credentials; the assistant only ever sees the link.

## What happens

1. A dry run shows you each platform's version.
2. A draft lets you check it on the platform before it goes live.
```

REPLY

```
“Publishing from an MCP client” — One markdown file, every platform, no copy and paste
- devto: draft → https://dev.to/ada/article-1 (post `pst_565d59c38bc4`)
```

Inkcast walkthrough, step 16, 2026-10-28T07:11:12Z. Recorded from the live run; credentials masked.

15-mcp-tool-call-inkcast-publish.png — inkcast_publish: a DEV draft with its URL and post id

**Step 17** — MCP inkcast_posts (mcp, 2026-10-28T07:11:12Z)

The post list shows the draft.

✓ inkcast_posts: the draft is listed

MODEL CONTEXT PROTOCOL, TOOLS/CALL

ok

## MCP tool call — inkcast_posts

The post list shows the draft.

CALL

```
inkcast_posts()
```

REPLY

```
Posts:
- [draft] 2026-10-28T07:11 devto: Publishing from an MCP client (post `pst_565d59c38bc4`) →
https://dev.to/ada/article-1
```

Inkcast walkthrough, step 17, 2026-10-28T07:11:12Z. Recorded from the live run; credentials masked.

16-mcp-tool-call-inkcast-posts.png — inkcast_posts: the draft is listed

### 4. Schedule a post, then cancel it

**Step 18** — MCP inkcast_publish (mcp, 2026-10-28T07:11:12Z)

A second story scheduled an hour ahead (Inkcast refuses anything under two minutes away).

✓ inkcast_publish: scheduled for the requested time

MODEL CONTEXT PROTOCOL, TOOLS/CALL

ok

**MCP tool call — inkcast_publish**

A second story scheduled an hour ahead (Inkcast refuses anything under two minutes away).

CALL

```
inkcast_publish({
  "markdown": "<markdown: 9 lines, shown below>",
  "publish_at": "2026-10-28T08:11:12+00:00"
})

markdown =
---
title: A story for later this hour
tags: [scheduling]
platforms: [devto]
---

# A story for later this hour

Scheduled from the MCP client, then cancelled before it went out.
```

REPLY

```
“A story for later this hour”
- devto: scheduled for 2026-10-28T08:11:12+00:00 (post `pst_fc8f71c396ee`)
```

Inkcast walkthrough, step 18, 2026-10-28T07:11:12Z. Recorded from the live run; credentials masked.

17-mcp-tool-call-inkcast-publish.png — inkcast_publish: scheduled for the requested time

**Step 19** — MCP inkcast_posts (mcp, 2026-10-28T07:11:12Z)

Listed as scheduled, next to the draft.

✓ inkcast_posts: the scheduled post is listed

MODEL CONTEXT PROTOCOL, TOOLS/CALL

ok

**MCP tool call — inkcast_posts**

Listed as scheduled, next to the draft.

CALL

```
inkcast_posts()
```

REPLY

```
Posts:
- [draft] 2026-10-28T07:11 devto: Publishing from an MCP client (post `pst_565d59c38bc4`) →
https://dev.to/ada/article-1
- [scheduled] 2026-10-28T08:11 devto: A story for later this hour (post `pst_fc8f71c396ee`)
```

Inkcast walkthrough, step 19, 2026-10-28T07:11:12Z. Recorded from the live run; credentials masked.

18-mcp-tool-call-inkcast-posts.png — inkcast_posts: the scheduled post is listed

**Step 20** — MCP inkcast_cancel (mcp, 2026-10-28T07:11:12Z)

Second thoughts: cancel it before it goes out.

✓ inkcast_cancel: the cancellation names the post




19-mcp-tool-call-inkcast-cancel.png — inkcast_cancel: the cancellation names the post

**Step 21** — MCP inkcast_posts (mcp, 2026-10-28T07:11:13Z)

Now listed as cancelled; nothing reached DEV.

✓ inkcast_posts: the post is listed as cancelled




20-mcp-tool-call-inkcast-posts.png — inkcast_posts: the post is listed as cancelled

### 5. A personal access token for CI

**Step 22** — MCP inkcast_create_token (mcp, 2026-10-28T07:11:13Z)

For CI the person needs a token the assistant never sees: another link to open.

✓ inkcast_create_token: a /tokens/new link with the name and scope

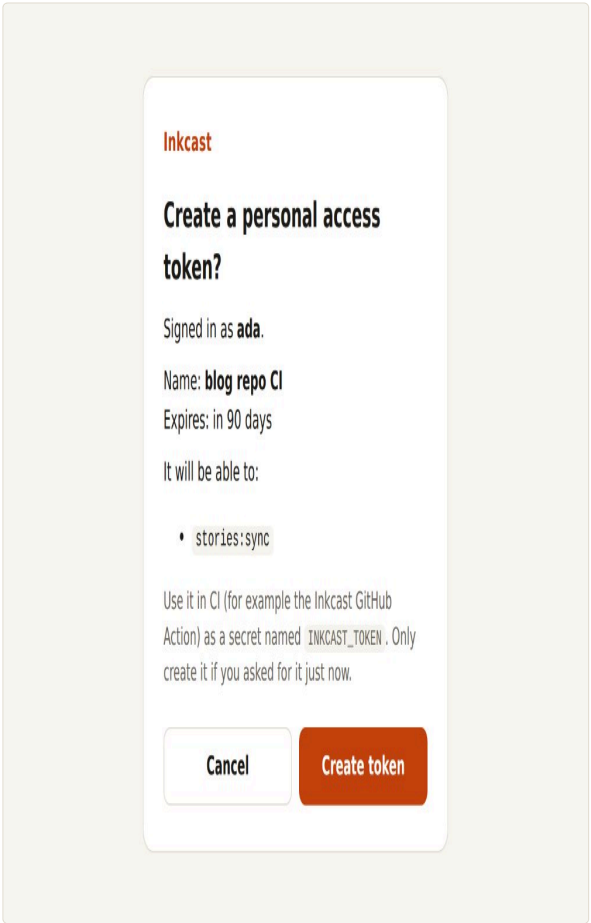



21-mcp-tool-call-inkcast-create-token.png — inkcast_create_token: a /tokens/new link with the name and scope

**Step 23** — token confirm (browser, 2026-10-28T07:11:13Z)

After signing in again, the person confirms the token's name, scope and expiry.

✓ http://localhost:8787/auth/consent?pending=D89Ctvw_m0y9WKE4O9nVeV5usFJ-Plvj — Create a personal access token?; the confirmation names the token and scope

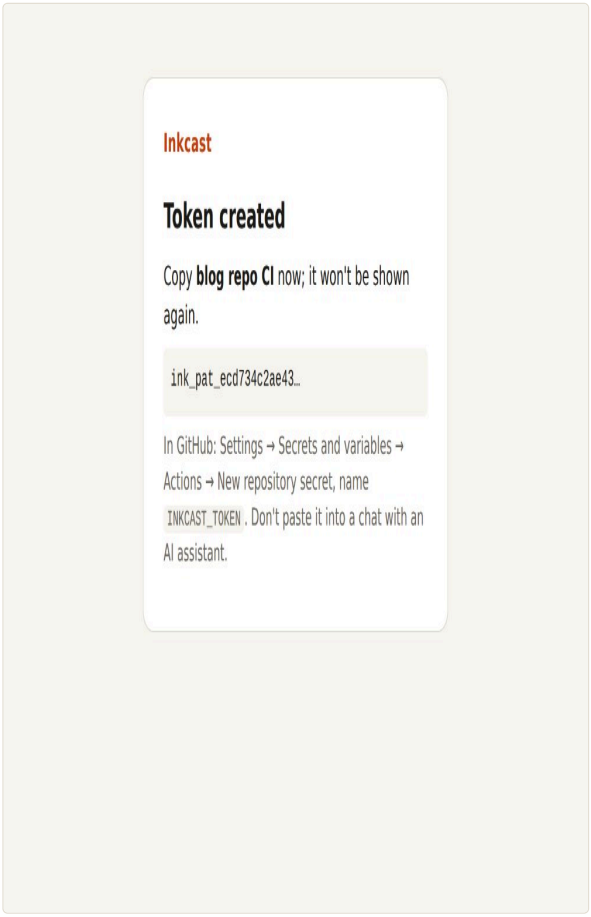



22-token-confirm.png — http://localhost:8787/auth/consent?pending=D89Ctvw_m0y9WKE4O9nVeV5usFJ-Plvj — Create a personal access token?; the confirmation names the token and scope

**Step 24** — token created (browser, 2026-10-28T07:11:14Z)

Shown once, to be stored as the CI secret INKCAST_TOKEN (masked here).

✓ http://localhost:8787/auth/consent — Token created; the token page, with the token masked




23-token-created.png — http://localhost:8787/auth/consent — Token created; the token page, with the token masked

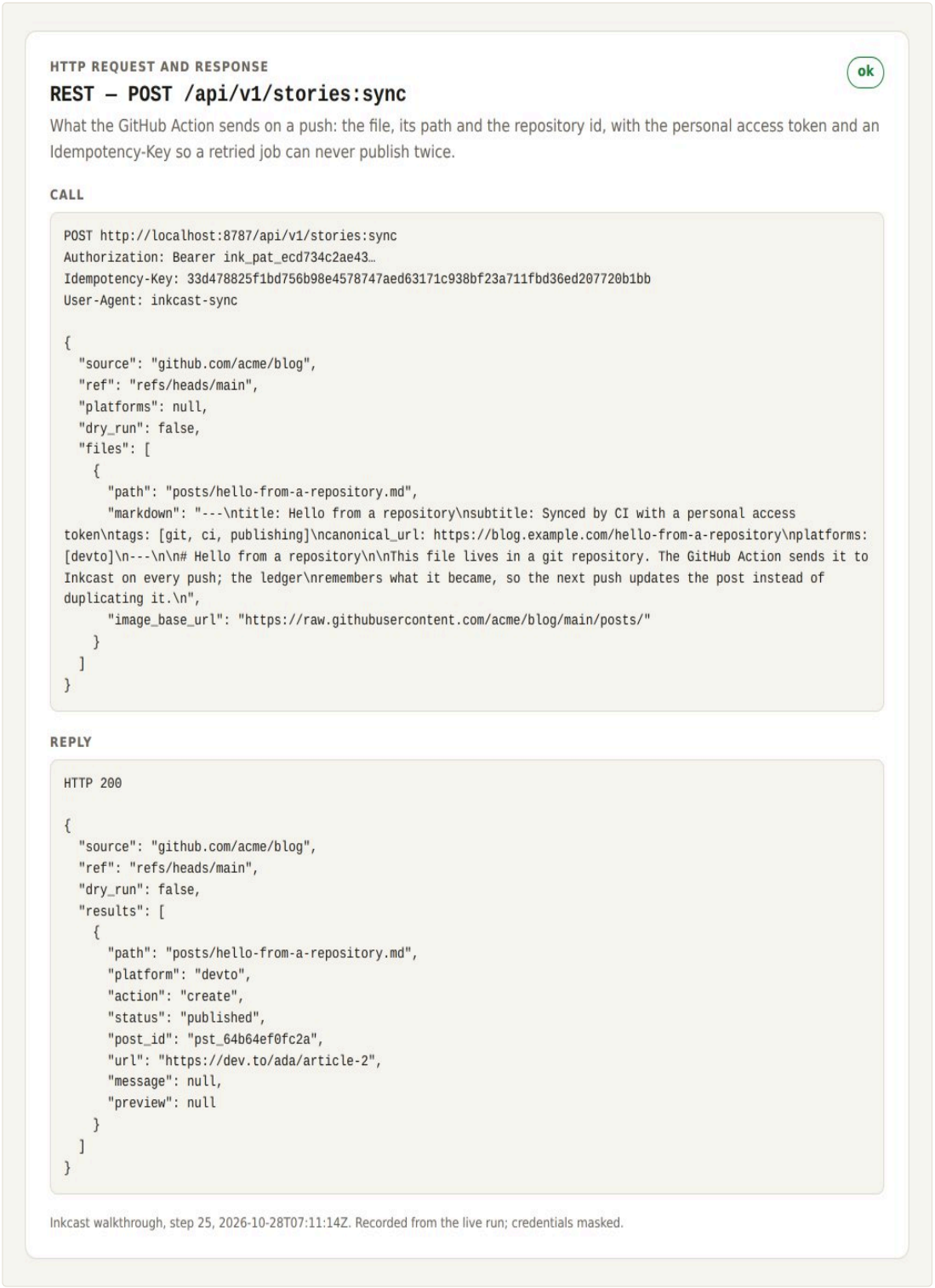
### 6. Git sync, as the GitHub Action does it

**Step 25** — REST — POST /api/v1/stories:sync (rest, 2026-10-28T07:11:14Z)

What the GitHub Action sends on a push: the file, its path and the repository id, with the personal access token and an

Idempotency-Key so a retried job can never publish twice.

✓ HTTP 200; the file was created on DEV with a URL

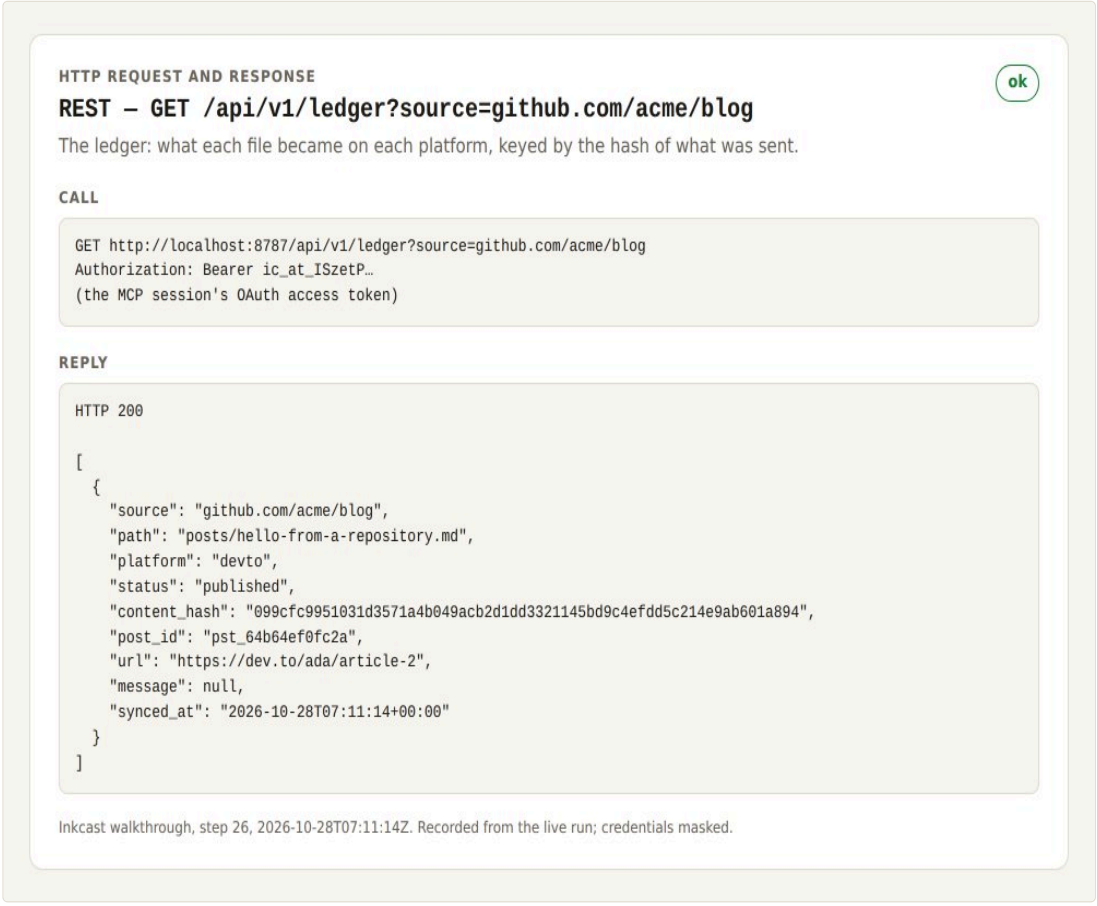



24-rest-post-api-v1-stories-sync.png — HTTP 200; the file was created on DEV with a URL

**Step 26** — REST — GET /api/v1/ledger?source=github.com/acme/blog (rest, 2026-10-28T07:11:15Z)

The ledger: what each file became on each platform, keyed by the hash of what was sent.

✓ HTTP 200; the ledger has the file, published, with a content hash




25-rest-get-api-v1-ledger-source-github-com-acme-blog.png — HTTP 200; the ledger has the file, published, with a content hash

### 7. Insights

**Step 27** — REST — GET /api/v1/insights/summary?days=7 (rest, 2026-10-28T07:11:16Z)

The person's own numbers, folded from domain events by the insights worker (settled after 2 poll(s)): posts by status, the ledger,

the sync outcomes, publish success and on-time rates.

✓ HTTP 200; the draft, the cancellation and the sync are all counted

HTTP REQUEST AND RESPONSE

ok

## REST — GET /api/v1/insights/summary?days=7

The person's own numbers, folded from domain events by the insights worker (settled after 2 poll(s)): posts by status, the ledger, the sync outcomes, publish success and on-time rates.

CALL

```
GET http://localhost:8787/api/v1/insights/summary?days=7
Authorization: Bearer ic_at_ISzetP…
(the MCP session's OAuth access token)
```

REPLY

```
HTTP 200

{
  "days": 7,
  "scope": "user",
  "scope_id": "usr_c519434b0d934140",
  "posts_by_status": {
    "cancelled": {
      "devto": 1
    },
    "draft": {
      "devto": 1
    },
    "published": {
      "devto": 1
    }
  },
  "ledger_by_status": {
    "published": 1
  },
  "publish_success": [
    {
      "platform": "devto",
      "numerator": 2.0,
      "denominator": 2.0,
      "rate": 1.0
    }
  ],
  "on_time": [],
  "sync_by_action": {
    "create": 1
  },
  "sync_by_platform": {
    "devto": 1
  }
}
```

Inkcast walkthrough, step 27, 2026-10-28T07:11:16Z. Recorded from the live run; credentials masked.

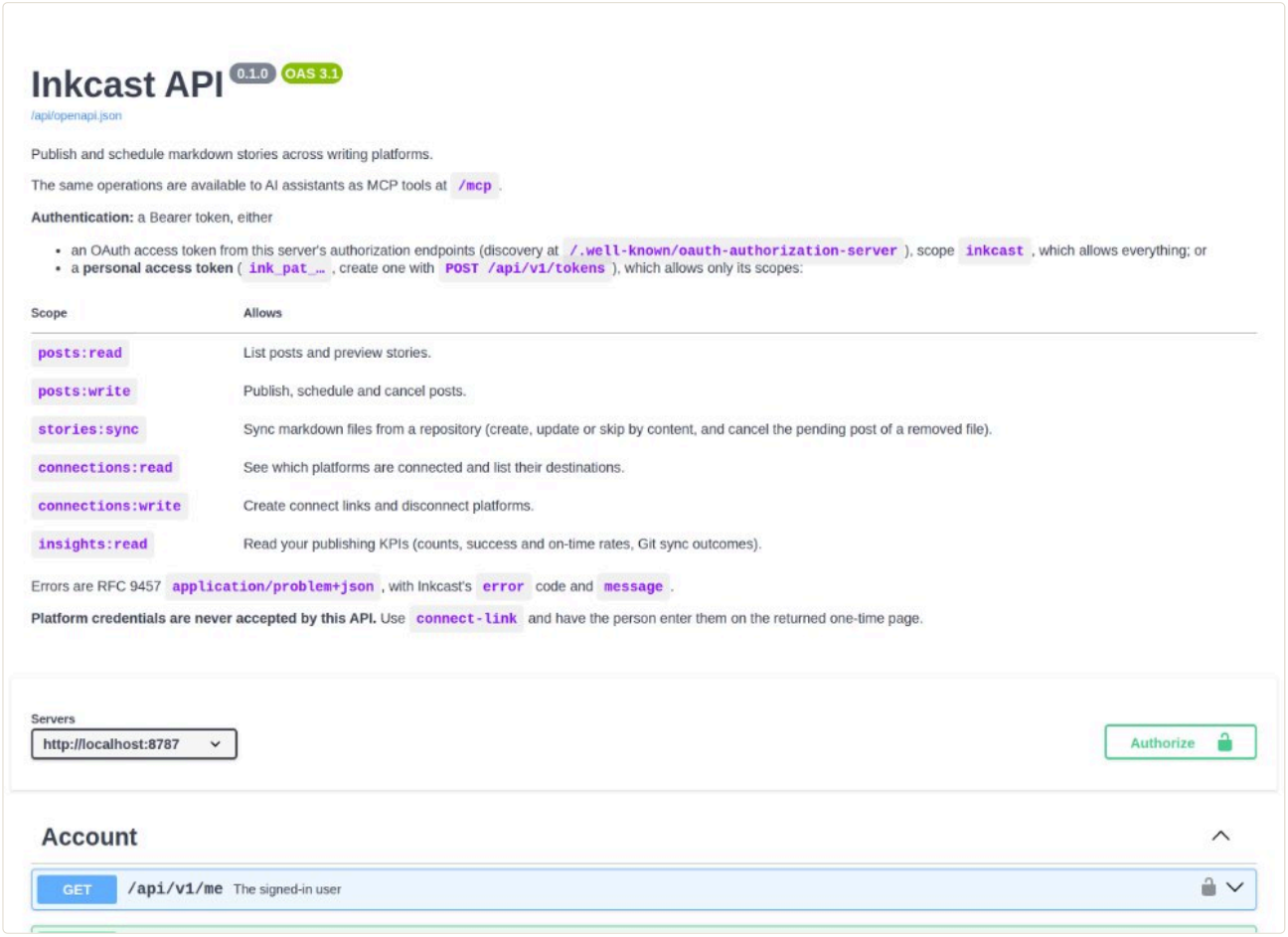
26-rest-get-api-v1-insights-summary-days-7.png — HTTP 200; the draft, the cancellation and the sync are all counted

### 8. The API reference

**Step 28** — api swagger (browser, 2026-10-28T07:11:17Z)

The REST reference, served by every deployment with vendored assets (nothing from a CDN).

✓ http://localhost:8787/api — Inkcast API 0.1.0 OAS 3.1; Swagger UI lists the stories:sync route

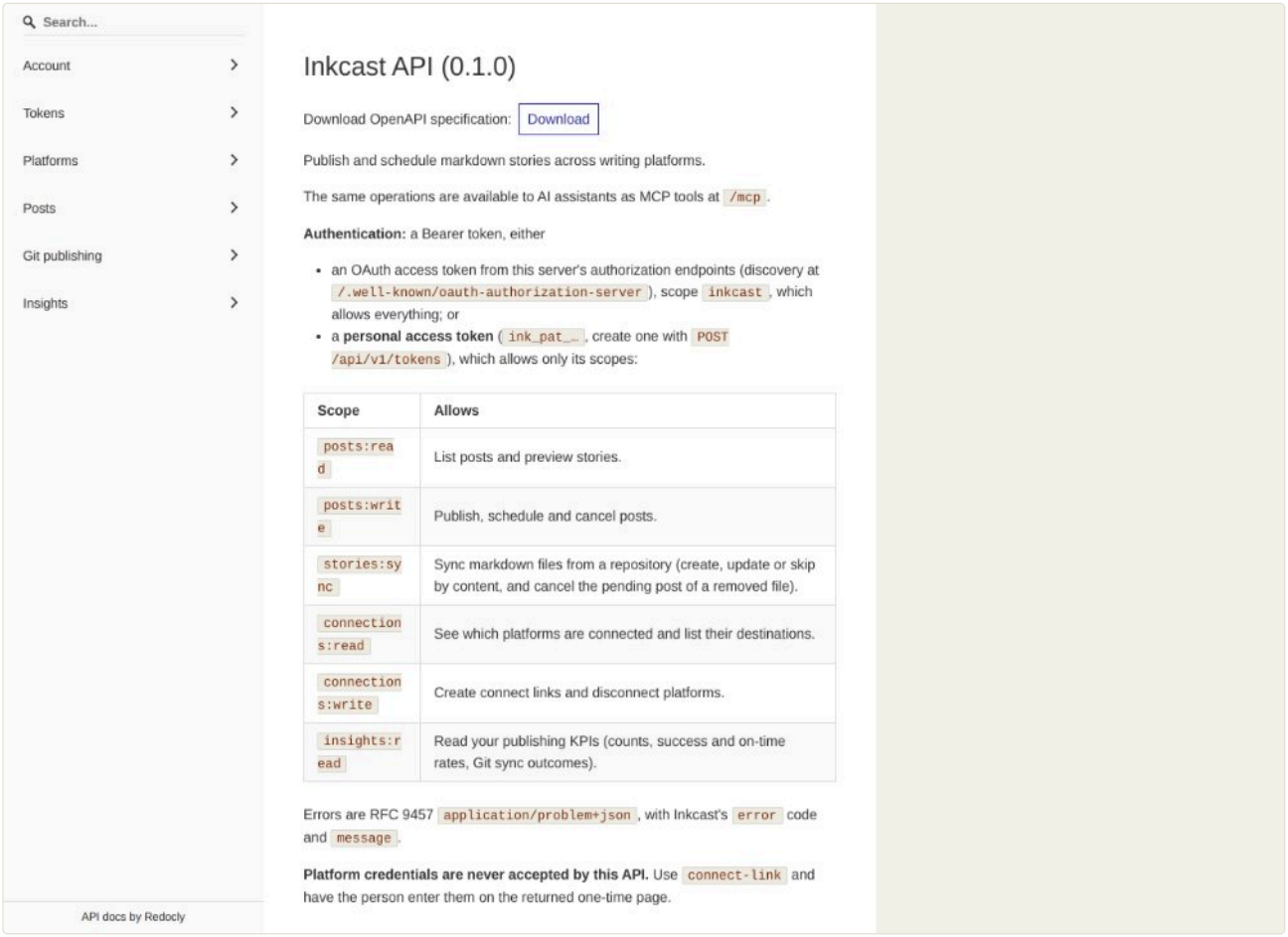



27-api-swagger.png — http://localhost:8787/api — Inkcast API 0.1.0 OAS 3.1; Swagger UI lists the stories:sync route

**Step 29** — api redoc (browser, 2026-10-28T07:11:18Z)

The same contract in ReDoc.

✓ http://localhost:8787/api/redoc — Inkcast API (0.1.0); ReDoc renders the API title




28-api-redoc.png — http://localhost:8787/api/redoc — Inkcast API (0.1.0); ReDoc renders the API title

## 9. What the platform received

**Step 30** — REST — GET /devto/api/articles/me/all (fake DEV) (rest, 2026-10-28T07:11:18Z)

Read back from the fake DEV, as the integrity check's witness: the draft and the synced file, and nothing for the cancelled

schedule.

✓ HTTP 200; two articles: the draft (unpublished) and the synced file (published)

HTTP REQUEST AND RESPONSE

ok

## REST — GET /devto/api/articles/me/all (fake DEV)

Read back from the fake DEV, as the integrity check's witness: the draft and the synced file, and nothing for the cancelled schedule.

CALL

```
GET http://127.0.0.1:8999/devto/api/articles/me/all
api-key: devt…
Accept: application/vnd.forem.api-v1+json
```

REPLY

```
HTTP 200

[
  {
    "id": 1001,
    "title": "Publishing from an MCP client",
    "canonical_url": "https://blog.example.com/publishing-from-an-mcp-client",
    "url": "https://dev.to/ada/article-1",
    "published": false,
    "body_markdown": "You write the story once, in markdown, and your assistant publishes it where you
asked.\nInkcast keeps the credentials; the assistant only ever sees the link.\n\n## What happens\n\n1. A dry
run shows you each platform's version.\n2. A draft lets you check it on the platform before it goes live.\n"
  },
  {
    "id": 1002,
    "title": "Hello from a repository",
    "canonical_url": "https://blog.example.com/hello-from-a-repository",
    "url": "https://dev.to/ada/article-2",
    "published": true,
    "body_markdown": "This file lives in a git repository. The GitHub Action sends it to Inkcast on every push;
the ledger\nremembers what it became, so the next push updates the post instead of duplicating it.\n"
  }
]
```

Inkcast walkthrough, step 30, 2026-10-28T07:11:18Z. Recorded from the live run; credentials masked.

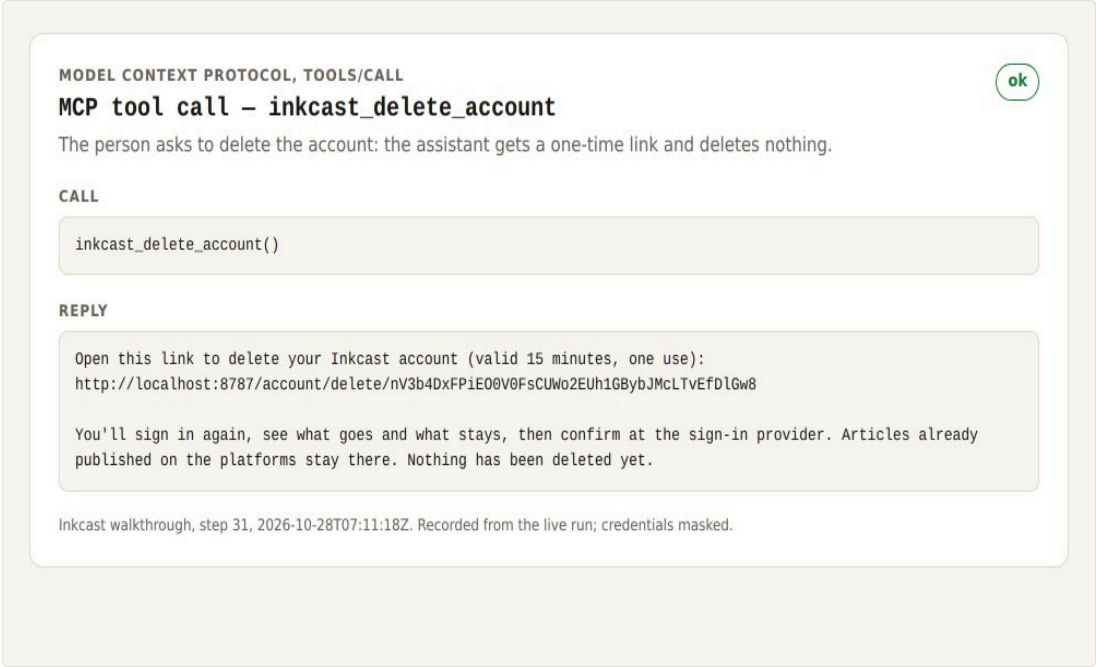
29-rest-get-devto-api-articles-me-all-fake-dev.png — HTTP 200; two articles: the draft (unpublished) and the synced file (published)

### 10. Delete my account

**Step 31** — MCP inkcast_delete_account (mcp, 2026-10-28T07:11:18Z)

The person asks to delete the account: the assistant gets a one-time link and deletes nothing.

✓ inkcast_delete_account: a one-time /account/delete/ link; nothing deleted yet

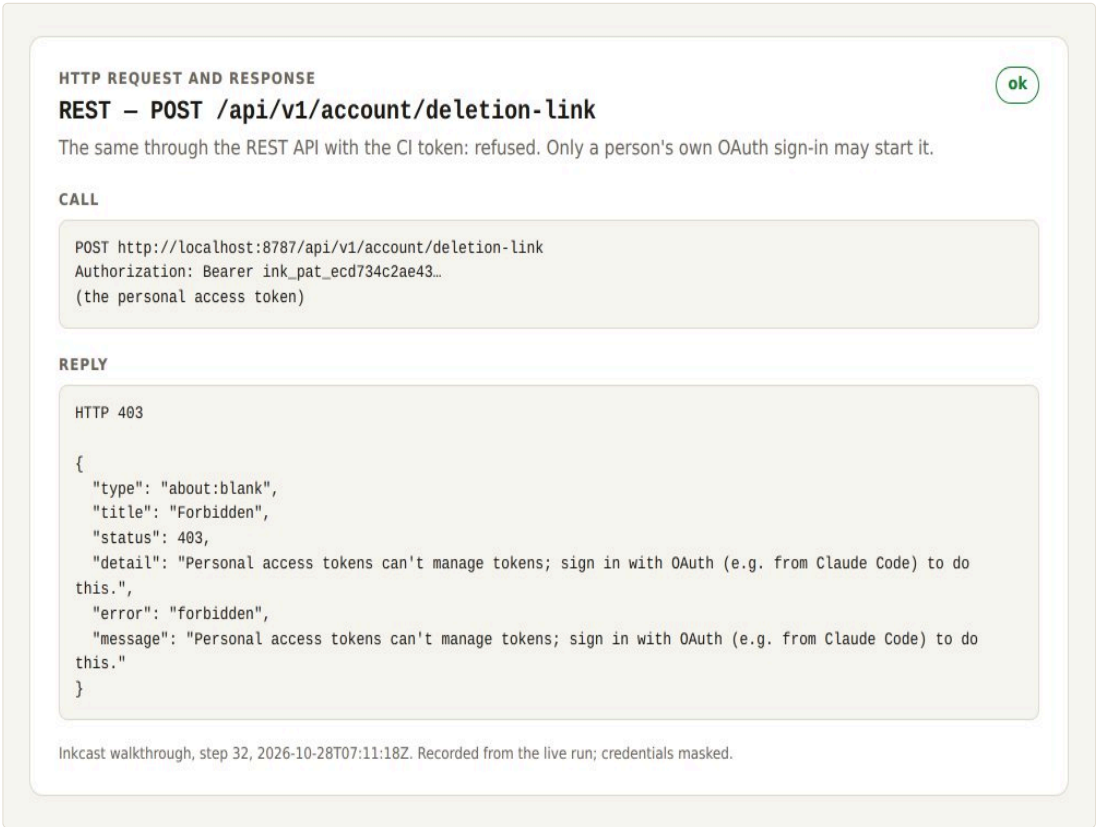



30-mcp-tool-call-inkcast-delete-account.png — inkcast_delete_account: a one-time /account/delete/ link; nothing deleted yet

**Step 32** — REST — POST /api/v1/account/deletion-link (rest, 2026-10-28T07:11:19Z)

The same through the REST API with the CI token: refused. Only a person's own OAuth sign-in may start it.

✓ HTTP 403; 403: a personal access token can't start an account deletion



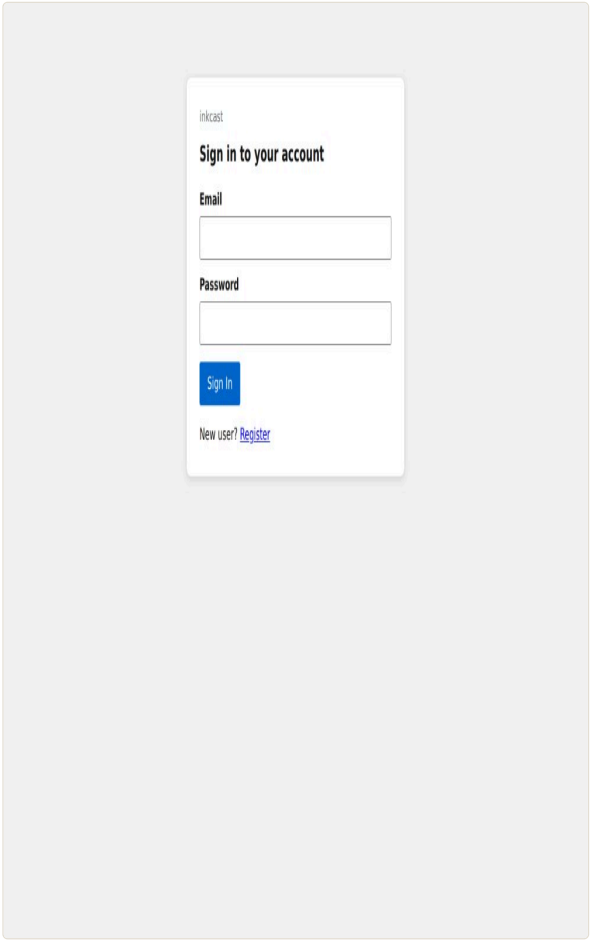

31-rest-post-api-v1-account-deletion-link.png — HTTP 403; 403: a personal access token can't start an account deletion

**Step 33** — deletion sign-in (browser, 2026-10-28T07:11:19Z)

The link asks for a fresh sign-in at the identity provider first.

✓ http://127.0.0.1:8999/kc/realms/inkcast/protocol/openid-connect/auth?response_type=code&client_id=inkcast&redirect_uri=http%3A%2F%2Flocalhost%3A8787%2Fauth%2Fcallback&scope=openid+email+pr

Rse4gbsiQDg_r25f5Cw&code_challenge=68H9mpKm8NenO2KVjY6peKlsxhApA7ACv_Dv2dUu2m0&code_challenge_method=S256 — Sign in to your account; the identity provider's sign-in form



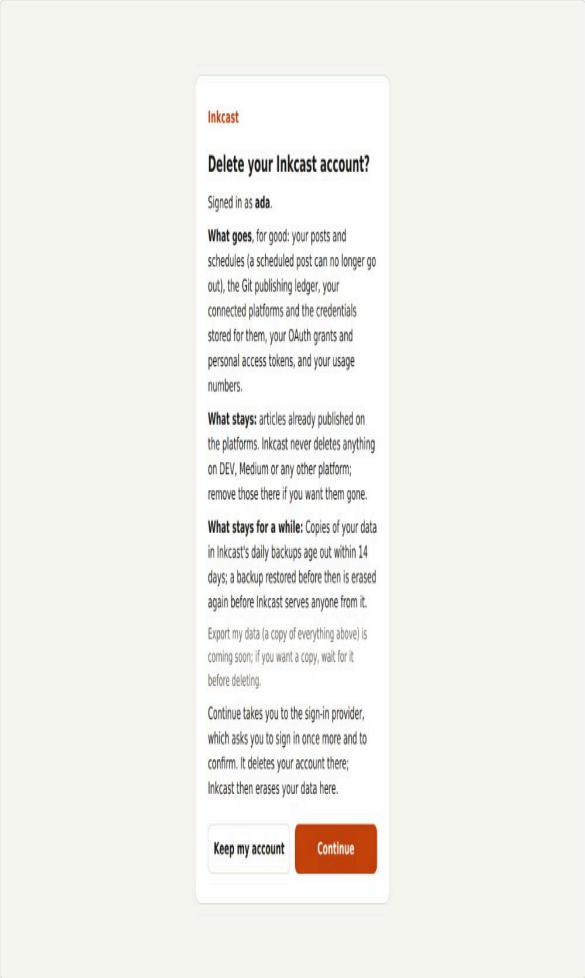

32-deletion-sign-in.png — http://127.0.0.1:8999/kc/realms/inkcast/protocol/openid-connect/auth?response_type=code&client_id=inkcast&redirect_uri=http%3A%2F%2Flocalhost%3A8787%2Fauth%2Fcallback&scope=openid+email+profile&state=1hdCDrWs Rse4gbsiQDg_r25f5Cw&code_challenge=68H9mpKm8NenO2KVjY6peKlsxhApA7ACv_Dv2dUu2m0&code_challenge_method=S256 — Sign in to your account; the identity provider's sign-in form

**Step 34** — deletion page (browser, 2026-10-28T07:11:20Z)

Inkcast's page: what goes (posts, schedules, the ledger, connections and credentials, tokens, usage numbers), what stays (articles already on the platforms), the export coming soon, and Continue.

✓ http://localhost:8787/auth/consent?pending=1hdCDrWsN3dNPCXyDsEP3I5iS6G8QIPN — Delete your Inkcast account?; what

goes, what stays, the export



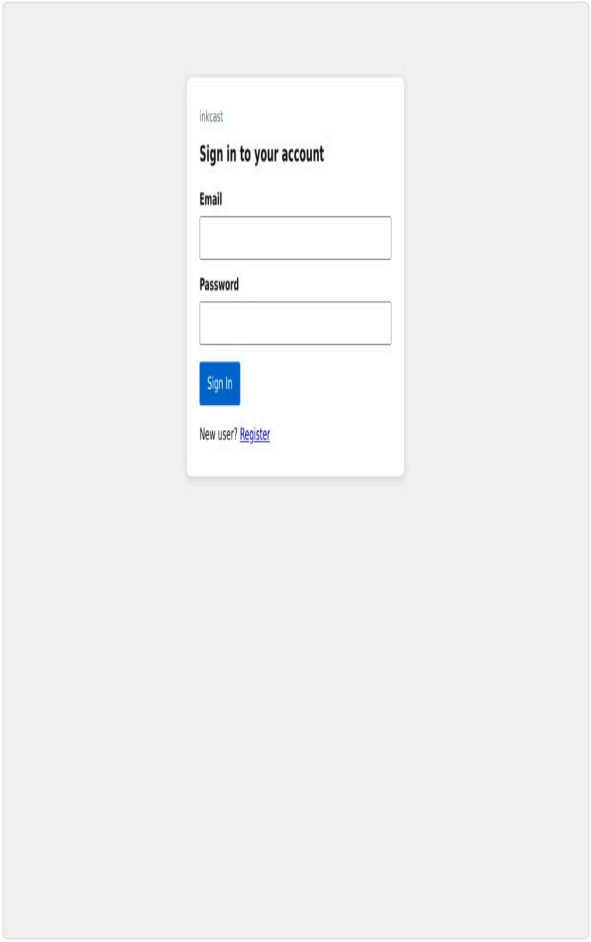

33-deletion-page.png — http://localhost:8787/auth/consent?pending=1hdCDrWsN3dNPCXyDsEP3I5iS6G8QlPN — Delete your Inkcast account?; what goes, what stays, the export

**Step 35** — provider re-authentication (browser, 2026-10-28T07:11:21Z)

The identity provider's delete_account action always asks for the password again.

✓ http://127.0.0.1:8999/kc/realms/inkcast/protocol/openid-connect/auth?response_type=code&client_id=inkcast&redirect_uri=http%3A%2F%2Flocalhost%3A8787%2Fauth%2Fcallback&scope=openid+email+pro
cjXf_Fu1vUMiOHwxKH1J3DcaXxW_LfM8pFM-

kU&code_challenge=nlKw02yMeo8R4hWkwVd3Pn3928N3Xdwy39opVzFpM8E&code_challenge_method=S256&kc_action=delete_account — Sign in to your account; the sign-in form once more



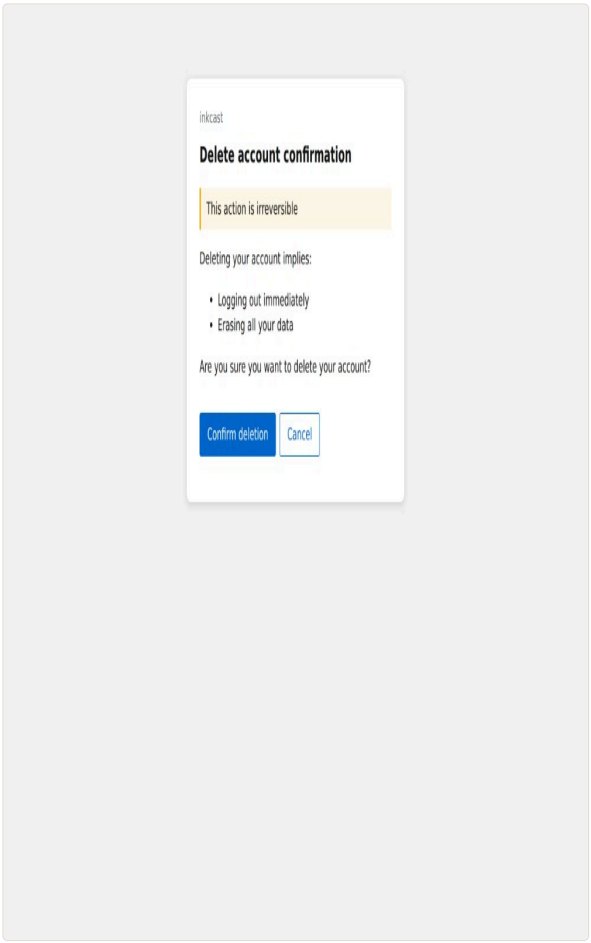

34-provider-re-authentication.png — http://127.0.0.1:8999/kc/realms/inkcast/protocol/openid-connect/auth?response_type=code&client_id=inkcast&redirect_uri=http%3A%2F%2Flocalhost%3A8787%2Fauth%2Fcallback&scope=openid+email+profile&state=yxgrB1bo:cjXf_Fu1vUMiOHwxKH1J3DcaXxW_LfM8pFM-kU&code_challenge=nlKw02yMeo8R4hWkwVd3Pn3928N3Xdwy39opVzFpM8E&code_challenge_method=S256&kc_action=delete_account — Sign in to your account; the sign-in form once more

**Step 36** — provider confirmation (browser, 2026-10-28T07:11:22Z)

The identity provider asks the person to confirm; Cancel would come back to Inkcast with nothing deleted.

✓ http://127.0.0.1:8999/kc/realms/inkcast/login-actions/authenticate?session_code=033250c5400e17c2&execution=e1&tab_id=t1

— Delete account confirmation; the delete-account confirmation



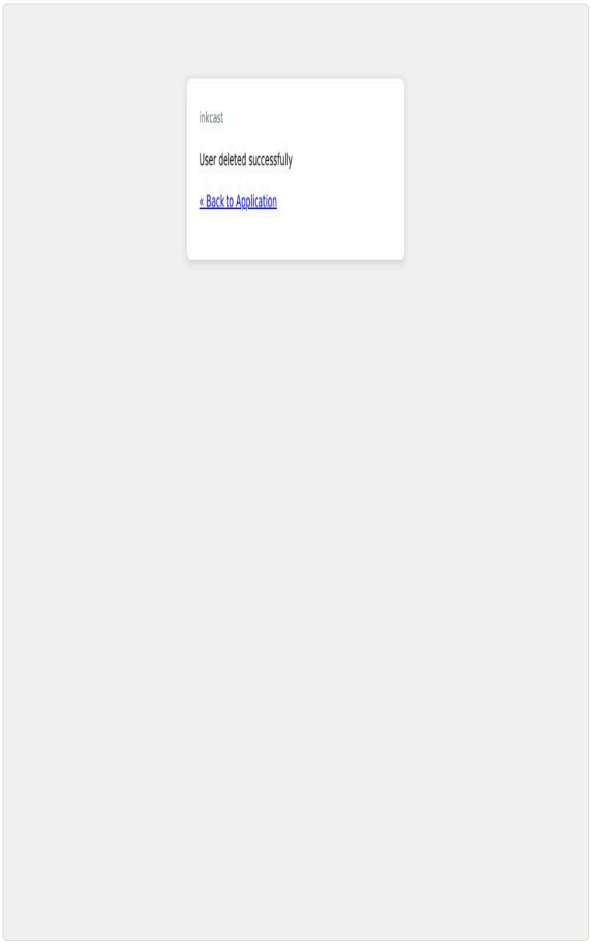

35-provider-confirmation.png — http://127.0.0.1:8999/kc/realms/inkcast/login-actions/authenticate?session_code=033250c5400e17c2&execution=e1&tab_id=t1 — Delete account confirmation; the delete-account confirmation

**Step 37** — provider deleted page (browser, 2026-10-28T07:11:23Z)

The provider deleted the account on the person's own authority and shows its own page; its Back to Application link is Inkcast's /account/return, which finishes the deletion.

✓ http://127.0.0.1:8999/kc/realms/inkcast/login-actions/required-action?

execution=delete_account&session_code=033250c5400e17c2 — rendered; deleted, with a link back



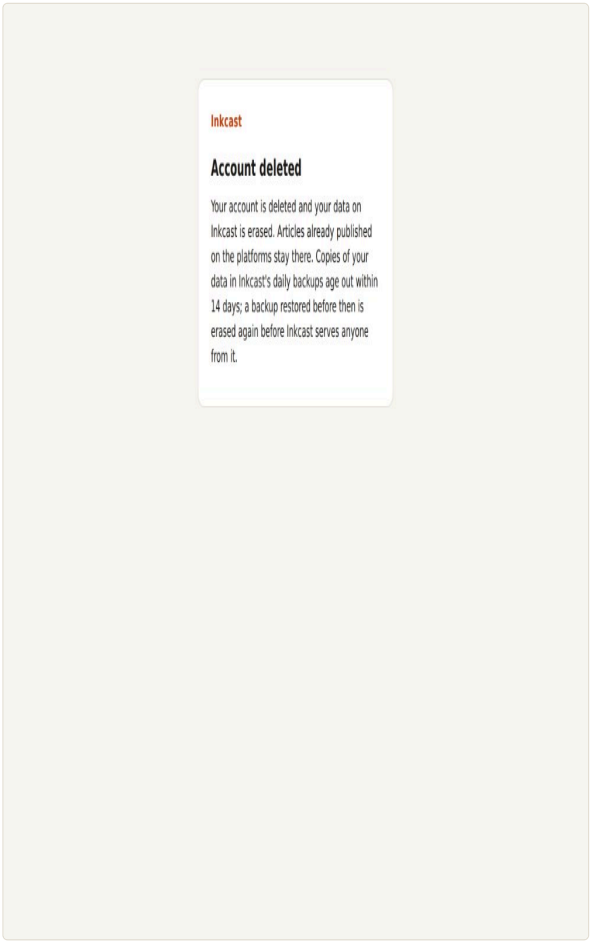

36-provider-deleted-page.png — http://127.0.0.1:8999/kc/realms/inkcast/login-actions/required-action?execution=delete_account&session_code=033250c5400e17c2 — rendered; deleted, with a link back

**Step 38** — account deleted (browser, 2026-10-28T07:11:24Z)

Back at Inkcast: it confirmed with the provider that the person is gone (read-only view-users), erased every row of theirs in one

transaction, and says so. Articles on DEV stay.

✓ http://localhost:8787/account/return — Account deleted; Inkcast says the account is deleted and the data erased




37-account-deleted.png — http://localhost:8787/account/return — Account deleted; Inkcast says the account is deleted and the data erased

**Step 39** — REST — GET /api/v1/me (rest, 2026-10-28T07:11:24Z)

The MCP session's OAuth token, after the deletion.

✓ HTTP 401; 401: the token is gone with the account




38-rest-get-api-v1-me.png — HTTP 401; 401: the token is gone with the account

**Step 40** — REST — GET /api/v1/me (rest, 2026-10-28T07:11:24Z)

The personal access token too.

✓ HTTP 401; 401: the CI token is gone with the account

HTTP REQUEST AND RESPONSE

ok

## REST — GET /api/v1/me

The personal access token too.

CALL

```
GET http://localhost:8787/api/v1/me
Authorization: Bearer ink_pat_ecd734c2ae43…
(the personal access token)
```

REPLY

```
HTTP 401

{
  "type": "about:blank",
  "title": "Unauthorized",
  "status": 401,
  "detail": "Send a valid Bearer token (an OAuth access token with the `inkcast` scope, or a personal access
token).",
  "error": "invalid_token",
  "message": "Send a valid Bearer token (an OAuth access token with the `inkcast` scope, or a personal access
token)."
}
```

Inkcast walkthrough, step 40, 2026-10-28T07:11:24Z. Recorded from the live run; credentials masked.

39-rest-get-api-v1-me.png — HTTP 401; 401: the CI token is gone with the account

**Step 41** — metric inkcast_identity_users_deleted_total (rest, 2026-10-28T07:11:24Z)

Operations see it as a number, never a name: one person erased, learnt of on the visit back from the provider (via=return) or from its DELETE_ACCOUNT event when the sync read that first (via=event).

✓ HTTP 200; the counter says 1 (via="return")

HTTP REQUEST AND RESPONSE

ok

## REST — GET /metrics (the metrics port)

Operations see it as a number, never a name: one person erased, learnt of on the visit back from the provider (via=return) or from its DELETE_ACCOUNT event when the sync read that first (via=event).

CALL

```
GET http://localhost:9090/metrics
```

REPLY

```
HTTP 200

# HELP inkcast_identity_users_deleted_total People erased from Inkcast once their account was gone, by how
Inkcast learnt of it (return, event, reconcile).
# TYPE inkcast_identity_users_deleted_total counter
inkcast_identity_users_deleted_total{via="return"} 1
(the other series not shown)
```

Inkcast walkthrough, step 41, 2026-10-28T07:11:24Z. Recorded from the live run; credentials masked.

40-rest-get-metrics-the-metrics-port.png — HTTP 200; the counter says 1 (via="return")

**Step 42** — REST — GET /devto/api/articles/me/all (fake DEV) (rest, 2026-10-28T07:11:24Z)

What stays: the fake DEV still holds both articles after the account is gone.

✓ HTTP 200; the same two articles as before

HTTP REQUEST AND RESPONSE

ok

## REST — GET /devto/api/articles/me/all (fake DEV)

What stays: the fake DEV still holds both articles after the account is gone.

CALL

```
GET http://127.0.0.1:8999/devto/api/articles/me/all
api-key: devt…
Accept: application/vnd.forem.api-v1+json
```

REPLY

```
HTTP 200

[
  {
    "id": 1001,
    "title": "Publishing from an MCP client",
    "canonical_url": "https://blog.example.com/publishing-from-an-mcp-client",
    "url": "https://dev.to/ada/article-1",
    "published": false,
    "body_markdown": "You write the story once, in markdown, and your assistant publishes it where you
asked.\nInkcast keeps the credentials; the assistant only ever sees the link.\n\n## What happens\n\n1. A dry
run shows you each platform's version.\n2. A draft lets you check it on the platform before it goes live.\n"
  },
  {
    "id": 1002,
    "title": "Hello from a repository",
    "canonical_url": "https://blog.example.com/hello-from-a-repository",
    "url": "https://dev.to/ada/article-2",
    "published": true,
    "body_markdown": "This file lives in a git repository. The GitHub Action sends it to Inkcast on every push;
the ledger\nremembers what it became, so the next push updates the post instead of duplicating it.\n"
  }
]
```

Inkcast walkthrough, step 42, 2026-10-28T07:11:24Z. Recorded from the live run; credentials masked.

41-rest-get-devto-api-articles-me-all-fake-dev.png — HTTP 200; the same two articles as before

## 3. Integrity of what was left behind

Checked from the files alone, after the servers were stopped (integrity_check.py): the article the fake DEV holds is the story the client published (title, and the sha256 of the body re-rendered through the server's own DEV renderer); the ledger's content hash equals the hash the server computes for the synced file; the cancelled schedule never reached DEV; no credential appears in full in any output file.

| Check | Result | Expected | Actual |
|---|---|---|---|
| DEV article: title of the draft | **pass** | `Publishing from an MCP client` | `Publishing from an MCP client` |
| DEV article: body of the draft (sha256) | **pass** | `8866e36398f42a875c1d8396b3c63493afd26a24325797ddddf0e3303fb72a3c` | `8866e36398f42a875c1d8396b3c63493afd26a24325797dddd` |
| DEV article: the draft is unpublished | **pass** | `false` | `false` |
| ledger: content_hash of the synced file | **pass** | `099cfc9951031d3571a4b049acb2d1dd3321145bd9c4efdd5c214e9ab601a894` | `099cfc9951031d3571a4b049acb2d1dd3321145bd9c4efdd5c2` |
| ledger: the synced file is published | **pass** | `published` | `published` |
| DEV article: body of the synced file (sha256) | **pass** | `60211cc4ac216304f2405bcdea991ef0e94612fd428a1ab426ab010d64d6bdc9` | `60211cc4ac216304f2405bcdea991ef0e94612fd428a1ab426a` |
| cancelled schedule: nothing on DEV | **pass** | `no article titled 'A story for later this hour'` | `[ "Publishing from an MCP client", "Hello from a re` |
| deletion: the OAuth token and the PAT answer 401 | **pass** | `[ 401, 401 ]` | `[ 401, 401 ]` |
| deletion: counted exactly once (via=return, or via=event when the sync read the event first) | **pass** | `inkcast_identity_users_deleted_total{via="return"\|"event"} 1, one series` | `[ "inkcast_identity_users_deleted_total{via=\"retur` |
| deletion: the articles on DEV outlived the account | **pass** | `[ "Publishing from an MCP client", "Hello from a repository" ]` | `[ "Publishing from an MCP client", "Hello from a re` |
| no credential in full in any output file | **pass** | `{}` | `{}` |

## 4. Environment

| Fact | Value |
|---|---|
| git_sha | `f4a9e53ced196b28244fdeee4f72200cf2587462` |
| git_ref | `main` |
| inkcast_version | `0.1.0` |
| python | `3.14.8` |
| playwright | `1.56.0` |
| mcp | `2.3.0` |
| chromium | `141.0.7390.37` |
| os | `Linux-6.17.0-1022-azure-x86_64-with-glibc2.39` |
| ci | `40648097581` |

Browser pages at 820×560 CSS px, device scale factor 2 (the API reference at 1280×900); panels at 1000 CSS px wide. The pictures in this document are downscaled copies; the originals are next to it.

## 5. Findings

No product finding was recorded for this run (no findings.json).

## 6. Files

| File | Bytes | What |
|---|---|---|
| 01-rest-get-well-known-oauth-protected-resource-mcp.png | 125,646 | picture |
| 02-rest-get-well-known-oauth-authorization-server.png | 216,420 | picture |
| 03-dynamic-client-registration.png | 198,785 | picture |
| 04-sign-in-page.png | 34,138 | picture |
| 05-consent-page.png | 94,971 | picture |
| 06-token-exchange.png | 110,773 | picture |
| 07-initialize-and-tools-list.png | 315,744 | picture |
| 08-mcp-tool-call-inkcast-platforms.png | 85,807 | picture |
| 09-mcp-tool-call-inkcast-connect.png | 111,383 | picture |
| 10-connect-page.png | 76,853 | picture |
| 11-dev-connected.png | 34,862 | picture |
| 12-mcp-tool-call-inkcast-platforms.png | 89,056 | picture |
| 13-mcp-tool-call-inkcast-destinations.png | 101,279 | picture |
| 14-mcp-tool-call-inkcast-publish.png | 267,373 | picture |
| 15-mcp-tool-call-inkcast-publish.png | 221,004 | picture |
| 16-mcp-tool-call-inkcast-posts.png | 88,902 | picture |
| 17-mcp-tool-call-inkcast-publish.png | 149,396 | picture |
| 18-mcp-tool-call-inkcast-posts.png | 105,519 | picture |
| 19-mcp-tool-call-inkcast-cancel.png | 86,832 | picture |
| 20-mcp-tool-call-inkcast-posts.png | 108,207 | picture |
| 21-mcp-tool-call-inkcast-create-token.png | 135,687 | picture |
| 22-token-confirm.png | 93,820 | picture |
| 23-token-created.png | 72,078 | picture |
| 24-rest-post-api-v1-stories-sync.png | 319,569 | picture |
| 25-rest-get-api-v1-ledger-source-github-com-acme-blog.png | 180,760 | picture |
| 26-rest-get-api-v1-insights-summary-days-7.png | 211,443 | picture |
| 27-api-swagger.png | 290,833 | picture |
| 28-api-redoc.png | 282,665 | picture |
| 29-rest-get-devto-api-articles-me-all-fake-dev.png | 261,077 | picture |
| 30-mcp-tool-call-inkcast-delete-account.png | 131,482 | picture |
| 31-rest-post-api-v1-account-deletion-link.png | 155,443 | picture |
| 32-deletion-sign-in.png | 44,086 | picture |
| 33-deletion-page.png | 258,382 | picture |
| 34-provider-re-authentication.png | 44,086 | picture |
| 35-provider-confirmation.png | 74,456 | picture |
| 36-provider-deleted-page.png | 34,967 | picture |
| 37-account-deleted.png | 90,196 | picture |
| 38-rest-get-api-v1-me.png | 139,778 | picture |
| 39-rest-get-api-v1-me.png | 136,900 | picture |
| 40-rest-get-metrics-the-metrics-port.png | 144,089 | picture |
| 41-rest-get-devto-api-articles-me-all-fake-dev.png | 250,568 | picture |
| env.json | 288 | data |
| fakes.log | 0 | process log |

| | | |
|---|---|---|
| integrity.jsonl | 2,381 | data |
| results.json | 48,663 | data |
| server.log | 20,054 | process log |